Atividade da Aula 11 — Testes a partir dos critérios de aceite

Valor: 0,25 (a segunda das quatro entregas que compõem o 1,0 de atividades de aula da Unidade 3)
Formato: produção em grupo (3 a 5 pessoas), **entrega individual** no Teams
Prazo: hoje, 05/11, até 23h59

> **É a continuação da Aula 10.** Hoje vocês testam **a história que cortaram em 29/10**: o PR de testes parte do PR da fatia e prova que o que foi construído lá funciona — e continua funcionando quando o banco for trocado na **Aula 12**. Os testes são os "testes derivados dos critérios de aceite, incluindo casos limite e de erro" que o **Trabalho 3** pede. A Unidade 3 termina em **03/12**, com a Prova 3 e a entrega do Trabalho 3 (branch `entrega-3`).

O que o grupo produz

Um **PR de testes**, entrado por revisão como qualquer mudança desde a Aula 6, sobre **a história da Aula 10**: a mesma do PR da fatia, e o **critério de aceite** de cada fatia, que está na descrição daquele PR e em `docs/analise.md`, é a fonte dos testes. Os testes vão em `tests/` e passam pela API, com `supertest` e `vitest`, como o `tests/doacoes.test.js` do template.

O teste que o PR da Aula 10 já trouxe **não conta como novo**: ele é o ponto de partida, e pode ser reforçado.

1. Os testes dos critérios

3 testes novos ou mais dos critérios de aceite da história. Cada um é um `it` — não um `it.todo` — e cumpre:

- o **nome do teste é o critério ou a regra** (não "teste 1", não "deve funcionar");
- o **Dado** prepara o estado pela própria API, o **Quando** é a ação e o **Então** confere o **valor** da resposta, não só o status;
- o Dado, o Quando e o Então aparecem à vista (por exemplo, em comentários) — se o critério já foi escrito assim na Aula 10, é só copiar.

Se só a primeira fatia está na `main`, os testes cobrem **o que o critério dela promete** (o efeito, o estado depois, o que o usuário vê) e **a regra de negócio** que ela implementa. Se o grupo já integrou outras fatias, os critérios delas entram também. Se um critério ainda não tem código, o teste vem primeiro e **fica vermelho**: implementem a fatia no mesmo PR, em commit à parte.

2. Dois casos limite ou de erro

2 testes novos ou mais do que o critério original **não previa**, cada um com o nome da regra e conferindo a **recusa**, a **mensagem** ou o **vazio** — não só o `200`. As quatro perguntas ajudam: *e se faltar algo, e se vier vazio, e se repetir, e se não existir?*

Se um desses testes ficar vermelho, o produto tem um buraco: corrijam-no **no mesmo PR, em commit à parte**, e digam isso na descrição.

3. A inversão de uma regra

Escolham **uma regra que a fatia da Aula 10 implementa** e que tenha teste, e façam, nesta ordem:

1. **Commitar** o que já funciona.
2. **Inverter a regra** no código (por exemplo, apagar a validação que a protege).
3. **Rodar a suíte inteira** e **contar os vermelhos**, sem corrigir nada antes da contagem.
4. **Desfazer** a inversão.

Se nenhum teste ficou vermelho, esse teste é decorativo: reforcem-no e repitam o experimento. O registro vai na descrição do PR, numa seção com exatamente este nome — a correção procura por ele:

```
## Inversão de regra

- Regra invertida: <qual regra, em que arquivo, o que foi alterado>
- Resultado: <N testes vermelhos: os nomes> — ou "nenhum", e o que o grupo fez depois
- Inversão desfeita: sim
```

A inversão é um experimento e **não entra no PR**: o código volta ao que era, e o CI do PR fica verde.

4. A ligação com a Aula 10 e o Uso de IA

A descrição do PR traz, além da inversão:

```
## História testada

- PR da fatia (Aula 10): <url>
- Critérios cobertos: <um por linha, com o nome do teste que o confere>
```

E a seção `## Uso de IA` preenchida: o que a IA gerou, o que ficou, o que vocês mudaram e por quê — ou "não usamos".

Além dos entregáveis

- **Cada integrante faz ao menos um commit hoje** — ou aparece como `Co-authored-by:` no commit do par. O histórico do repositório é o que comprova participação.

- **A suíte verde na `main`** e o **PostgreSQL respondendo**. **Não valem ponto hoje**, mas a Aula 12 troca o banco com esta suíte como prova: quem chegar lá sem suíte verde não tem como mostrar que o comportamento ficou.
- **CI verde** e a `main` **protegida**.

Como entregar

No Teams, na atividade da Aula 11, cole exatamente estas três linhas:

```
Repositório: <url do repositório do grupo>
PR dos testes: <url do PR de testes>
Meu commit de hoje: <url de um commit seu, feito hoje>
```

O PR pode estar **aberto ou integrado**. A produção é do grupo; a entrega é individual, porque a nota é individual.

Como é corrigido

Binário: entregou no prazo e no formato, pontuou. Não há nota parcial. A conferência é de presença e de forma, e a lista abaixo é exaustiva; a qualidade é avaliada no Trabalho 3.

Perde o ponto quem: entrega fora do prazo · manda o link do repositório sem o link do PR · entrega um PR com **menos de 3 testes novos de critério** (`it.todo` não conta) · com **menos de 2 testes novos de limite ou de erro** · sem a seção `## Inversão de regra` na descrição, ou com ela **sem a regra invertida**, **sem o resultado** (os nomes dos testes vermelhos, ou "nenhum" e o que o grupo fez) ou **sem "inversão desfeita"** · sem a seção `## História testada` , ou com ela **sem o link do PR da fatia da Aula 10** · entrega um PR com o **CI vermelho** · com o `Uso de IA` sem preencher · não tem commit próprio de hoje (a conferência é individual: quem não tem commit seu perde o ponto).

A qualidade é avaliada depois, e vale muito mais: "testes cobrem os critérios de aceite das histórias implementadas" é critério de aceite do **Trabalho 3** (2,0 do artefato). Na **defesa individual** (1,0) em **03/12**, cada integrante faz uma pequena modificação ao vivo e responde sobre uma parte que não escreveu — e saber o que cada teste protege é o caminho até ela.

Nível de IA nesta atividade: IA como colaboradora

Pode: escrever o rascunho dos testes, sugerir casos limite, explicar uma mensagem de erro do `vitest` .

Antes de aceitar os testes gerados: (1) para cada `expect` , **o que precisa quebrar para ele falhar?** (2) o Dado prepara o estado, ou o teste depende de um dado que já estava lá? (3) o teste confere o **valor** ou só o status? (4) o nome diz a regra? (5) a inversão da regra deixa algum teste vermelho?

Continua sendo de vocês: o critério, a decisão do que o produto deve fazer em cada limite, a escolha da regra a inverter e a explicação de cada teste. Em qualquer nível, vocês continuam responsáveis por **verificar, testar, corrigir e defender** o resultado — e **“a IA escreveu assim” não é justificativa aceita**.